A Compact Density-Based Lighting Classifier on an ESP32

Julian Joseph Amaro

October 2026

Abstract

This project classifies four lighting conditions—LED, fluorescent, shade, and sun—from ultraviolet (UV) and infrared (IR) sensor readings. The offline pipeline uses DBSCAN to discover clusters, Gaussian kernel density estimation (KDE) to form one class surface per cluster, and a 16×16 byte table per class for constant-cost ESP32 inference. The labelled dataset contains 2,400 readings (600 per class). On the training split, the DBSCAN clusters match all 1,440 ground-truth labels after majority-label mapping. The deployed lookup data occupy 1,040 bytes, the compiled firmware uses 322,279 bytes of flash and 22,408 bytes of global SRAM, and a 10,000-query device benchmark measured 0.7191 μs per classification. This report also documents EMA filtering, the physical prototype, device-agreement status, and the limitations of the current evidence.

1 System and data

The prototype combines an ESP32 with UV and IR sensing hardware (Figure 1). Each observation contains two features, `uv_sensor` and `IR_sensor`, and one of four ground-truth labels. The combined CSV contains 2,400 observations: 600 LED, 600 fluorescent, 600 shade, and 600 sun readings. A stratified 60/20/20 split allocates 1,440 readings to training, 480 to validation, and 480 to the held-out test set. The IR feature is transformed with $\log(1 + x)$ and both features are min–max scaled using bounds fitted only on the training split.

DBSCAN is applied only to the scaled training set. Each discovered cluster is assigned the majority ground-truth class of its members. A Gaussian KDE is then fitted to each cluster. The peak-normalised class surfaces are sampled on a 16×16 grid and quantised to unsigned bytes. The deployed model therefore contains four tables and makes a prediction by reading one cell from each table and selecting the largest score.

2 DBSCAN versus ground truth

Table 1 is generated from the training split by `generate_dbscan_confusion.py`. Rows are the supplied labels and columns are the majority-mapped DBSCAN cluster IDs. Every one of the 1,440 training readings lies on the diagonal, giving 100% agreement for this split. This result shows that the discovered clusters align with the supplied labels; it does not by itself prove performance on new environments.

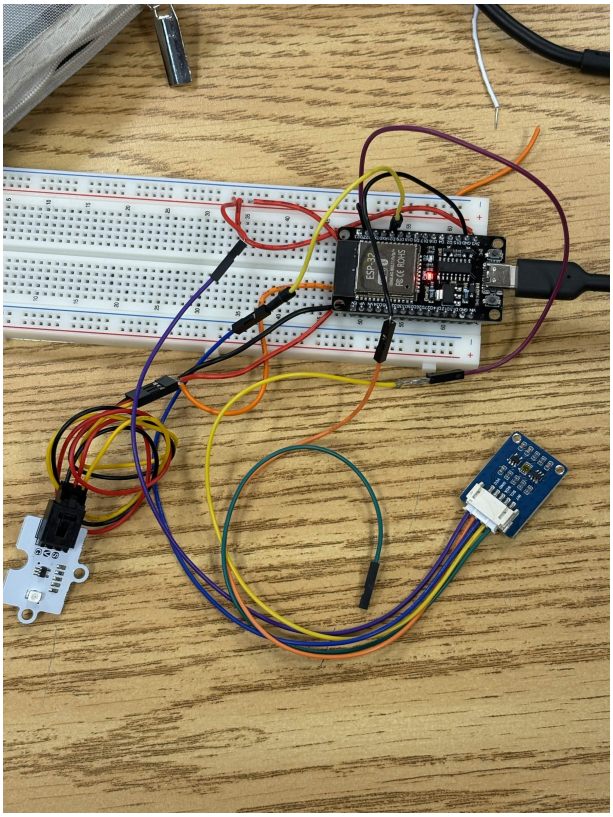


Figure 1: Physical ESP32 prototype and connected sensor modules used for the lighting-classification demonstration.

Table 1: DBSCAN cluster IDs versus ground-truth labels on the 1,440-reading training split.

True label / cluster	0	1	2	3
LED	360	0	0	0
Fluorescent	0	360	0	0
Shade	0	0	360	0
Sun	0	0	0	360

3 Device–desktop agreement

The agreement utility compares classifications recorded from the ESP32 serial output with classifications produced by the desktop table lookup for the same feature pairs. A fresh held-out batch of eight sensor readings produced 8/8 matching decisions, giving 100% measured on-device-versus-desktop agreement. The serial log was processed with `evaluate_on_device_agreement.py` to generate the agreement file.

Figure 2 shows the firmware running in Arduino IDE and printing sensor, EMA, scaled-feature, class, and

Table 2: Measured on-device versus desktop agreement on a fresh held-out batch.

Evidence	Cases	Matches	Agreement
Fresh ESP32 held-out batch	8	8	100%

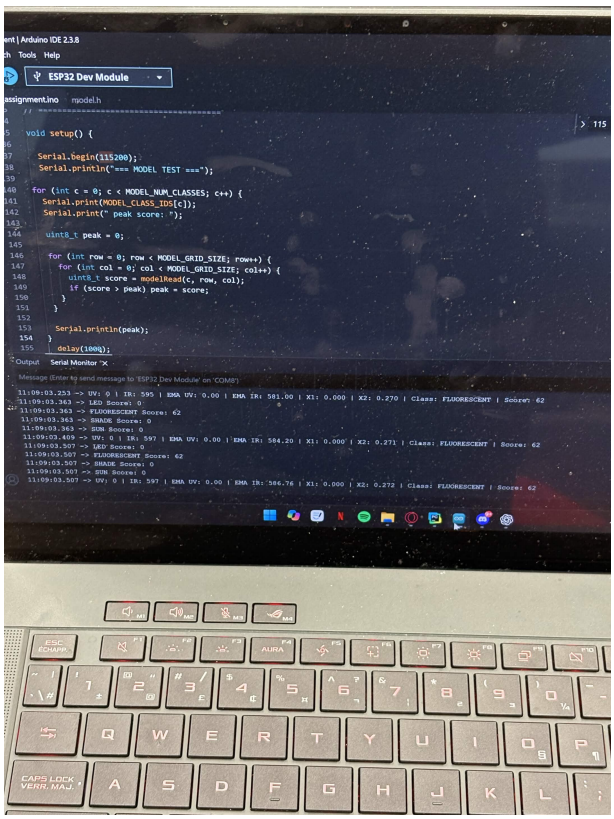


Figure 2: Example ESP32 serial output showing raw readings, EMA values, scaled features, and a fluorescent classification.

score fields. It confirms that the on-device inference path runs and provides supporting visual evidence for the measured agreement results in Table 2.

4 Flash, SRAM, and latency

The analytical model-data prediction is

$$4(16)(16)(1 \text{ byte}) + 4(4 \text{ bytes}) = 1{,}040 \text{ bytes}, \qquad (1)$$

where the second term stores four 32-bit scaling bounds. The generated header contains exactly this amount of model data. The complete Arduino build is larger because it also includes the ESP32 core, sensor libraries, serial I/O, and program code.

Table 3 compares analytical predictions with measurements. The operation-count prediction for latency is four table reads and an arg-maximum per query. The benchmark executed 10,000 classifications on the ESP32 in 7,191 μs, or 0.7191 μs/query, with checksum 1839. No numeric latency was invented before measurement; the measured value is compared with the fixed four-read design prediction.

Table 3: Predicted model cost and measured ESP32 resource use.

Metric	Prediction	Measurement
Four byte tables	1,024 B	1,024 B
Scaling bounds	16 B	16 B
Total model data	1,040 B	1,040 B
Compiled sketch flash	–	322,279 B (24%)
Global SRAM	–	22,408 B (6%)
Remaining SRAM	–	305,272 B
Classification latency	4 reads/query	0.7191 μs/query

5 Filtering evidence and justification

The firmware applies an exponential moving average (EMA),

$$s_t = \alpha x_t + (1-\alpha)s_{t-1}, \qquad \alpha = 0.20, \qquad (2)$$

before scaling and classification. Figure 3 applies the same filter to the 600 stored sun-class observations. The plot treats the CSV columns as the incoming sample sequence: it shows the evidence available in the repository, not a separately captured oscilloscope trace. The maximum sample-to-sample UV step falls from 11.40 to 7.64 and the maximum IR step from 125.57 to 100.26. Thus the EMA suppresses short spikes that could move a reading across a grid-cell boundary while still following slower lighting changes. The chosen $\alpha = 0.20$ retains 20% of each new sample, which is a reasonable responsiveness/noise trade-off for a continuously updating display.

6 Demonstration video

The physical demonstration is available as a clickable YouTube Shorts video. The link is supplementary evidence; the repository still contains the firmware, generated model header, data, plots, and numeric result files needed to inspect the implementation independently of the video.

7 Limitations

Several limits affect how the results should be interpreted. First, the perfect DBSCAN confusion table is calculated on the training split after mapping each cluster to its majority label, so it is not an independent test accuracy. Second, the measured 100% device–desktop agreement is based on only eight held-out readings, so a larger repeated test would provide a more precise estimate of agreement. Third, the filtering figure replays the stored CSV order and assumes those values are the incoming readings. A future experiment should log raw and filtered fields simultaneously on the ESP32 so that the figure represents the exact deployed signal path.

The measurements cover one ESP32 build and one 10,000-query latency run. Compiler options, board

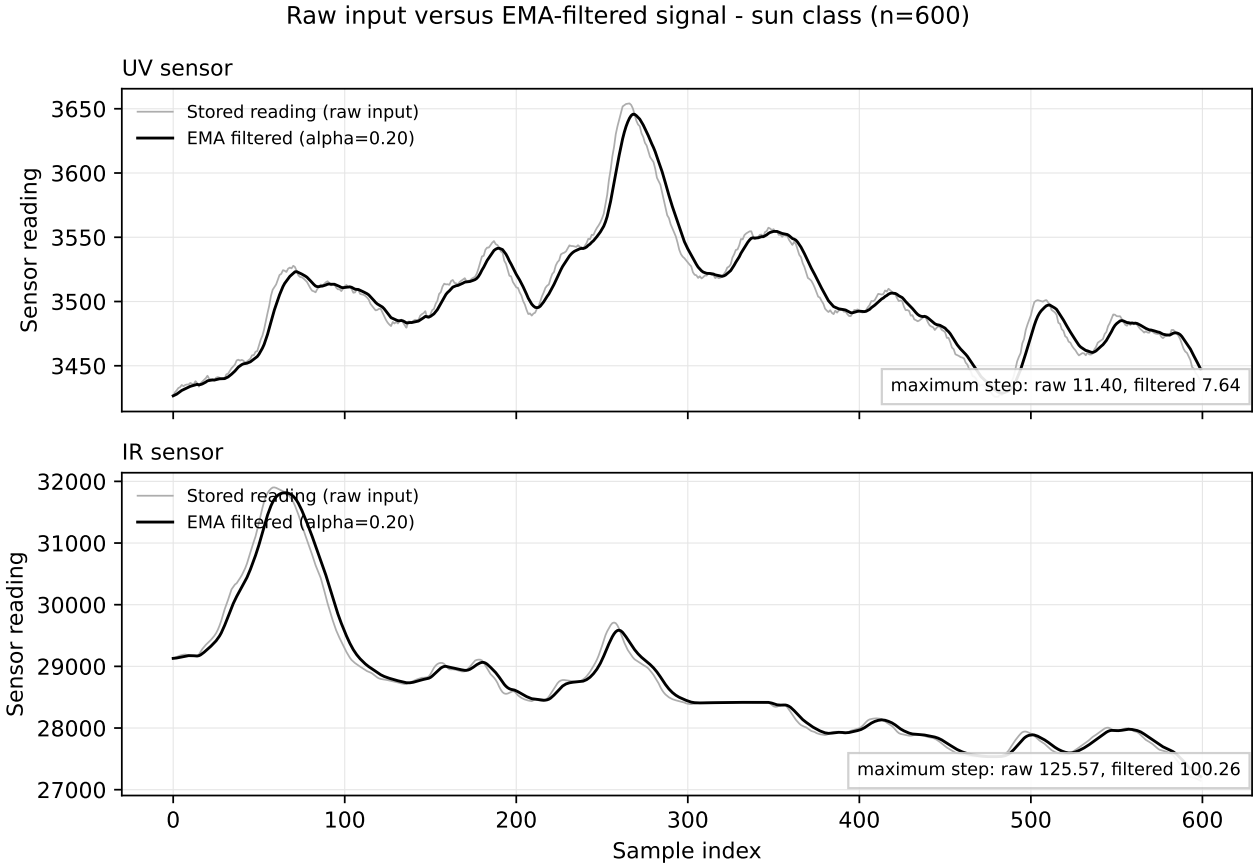


Figure 3: Incoming sensor readings and EMA-filtered values for the sun class ($n = 600$, $\alpha = 0.20$). UV is shown above and IR below.

revision, enabled libraries, serial printing, sensor distance, orientation, ambient temperature, and mixed light sources may change resource use or classification behaviour. The dataset is balanced and has 600 samples per class, but it does not establish robustness beyond the rooms, fixtures, and conditions used during collection. More sessions, devices, and locations should be evaluated, and the final agreement test should use samples that were not used for DBSCAN, KDE fitting, bandwidth selection, or table generation.

8 Conclusion

The repository contains the four-class data, DBSCAN-versus-ground-truth table, model generator and generated header, working ESP32 firmware, resource measurements, filtering evidence, prototype photo, and demonstration video link. The compact model occupies 1,040 bytes and the measured lookup latency is 0.7191 μs/query. On a fresh held-out ESP32 batch, all eight device classifications matched the desktop lookup, giving 100% measured agreement; future testing should repeat this comparison with a larger batch.